

MQ Panel

Complete Change Log · V1.0 → V1.3

TrendArchitectMQPanel is an open-source NinjaTrader 8 ChartTrader panel indicator that arms automated market entries on TrendArchitect MQB and MQS signal dots, combined with a full suite of trade management tools, profit trailing, and session risk management — all in a dark-terminal UI inspired by the DailyRangeBot design system.

Contributor	Contribution
Alighten	Button panel base — AlightenButtonPanelV0004
_Jason / B3AR	TrendArchitect indicator
Khanh	DailyRangeBot UI design system
Spoobie	V1.1 production hardening

V1.0 — Initial Release

Foundation release

Signal Detection

- ARM MQB and ARM MQS toggle buttons — arm a long or short market entry on the next MQB/MQS dot
- Entry quantity read live from the ChartTrader quantity selector
- Signal detection scans `DrawObjects` across all chart indicators
- Previous-bar lookback safety net catches signals drawn just after the panel's tick fired
- Session-open guard prevents stale dots triggering entries on load

Arm Modes

- Opposite Signal mode — **Off**, **Close**, or **Reverse** when opposing position exists, switchable live
- After Entry mode — **Disarm**, **ReArm**, **ReArmMQB**, or **ReArmMQS**, switchable live

Trade Management

Full suite carried from Alighten V0004: BE+N (x2), Price+N, Entry+N, Bracket, Add Stop, Half, Double, Naked, Split, Flatten Everything.

UI Design

- DailyRangeBot dark terminal palette (`C_BG`, `C_CARD`, `C_BORDER`, `C_GREEN`, `C_RED`, `C_AMBER`, `C_BLUE`, `C_PURPLE`)

- Card layout, colored-border buttons, Consolas font, section dividers
- ScrollView wrapper with fixed MaxHeight
- Self-contained — enums nested inside the class, no cross-version dependencies

V1.1 — Production Hardening

Contributed by Spoobie

Change	Description
<code>OrderEntry.Manual</code>	All order submissions changed from Automated to Manual — matches ChartTrader button behavior, avoids rejections on live brokerage connections
<code>SubmitPanelOrders()</code>	Centralized submission helper — validates account connection status before every <code>Submit()</code> call
<code>ValidateAccountForTrading()</code>	Pre-flight guard — checks account non-null and <code>ConnectionStatus == Connected</code>
Quantity selector rework	4-tier fallback: <code>QuantityUpDown</code> selector → <code>QuantityEdit</code> <code>TextBox</code> → <code>lastKnown</code> cache → <code>ChartTrader.Quantity</code> property
<code>WireChartTraderQuantitySelector()</code>	Attaches <code>ValueChanged</code> event listener at panel creation — no polling needed
<code>DetachChartTraderQuantitySelector()</code>	Cleanly unhooks events on disposal — prevents memory leaks
<code>ResolveAccount()</code>	Returns <code>SelectedAccount</code> directly — no fragile display-name string matching
<code>ResolveInstrument()</code>	Falls back to indicator's own <code>Instrument</code> property if selector lookup returns null
<code>GetChartTraderWindow()</code>	Null-safe helper extracted from repeated <code>Window.GetWindow()</code> call sites
ARM button re-arm reset	<code>lastMqbEntryBar</code> / <code>lastMqsEntryBar</code> reset to -1 on manual re-arm — fresh detection guaranteed
Bar stamp unconditional	<code>lastEntryBar</code> stamped on every detection regardless of <code>ReArm</code> mode — prevents <code>OnEachTick</code> re-entry on same dot
Generated code block	Re-added with correctly typed enum parameters (<code>OppositeSignalMode</code> , <code>ReArmMode</code>)

V1.2 — Collapsible Sections, Persistent State & Layout

Major UX release

Collapsible Sections

- Every section has a clickable header row with a chevron (▣/▼) that toggles content Visible/Collapsed
- Label row always stays visible — user can always re-expand

- 8 NinjaScriptProperty bools store open/closed state — survive chart reloads, workspace saves, template saves
- Browsable(false) keeps state properties out of the properties dialog

Default Collapse State

Section	Default
Signal Arming	Expanded
Opposite Signal	Expanded
After Entry	Expanded
Trade Management	Collapsed
All sub-sections	Collapsed

Other V1.2 Changes

- **Rebuild Panel button** (■) — sits left of version chip in header; calls DisposeWPFControls then CreateWPFControls for live rebuild
- **ButtonHeight property** — range 18–40, default 26, under Panel Layout group. Takes effect on rebuild
- **Trade Management grouping** — Breakeven, Targets, Bracket/Stop, Size nested as sub-sections inside one parent collapsible
- **C_LABEL color** — new palette entry Color.FromRgb(120,128,148); replaces C_DIM which washed out on many NT skins
- **Labels bolded** throughout for readability across all NinjaTrader skin themes
- **Rebuild button fix** — Style=null strips NT default button style that inflated the header row height

V1.2.1 — Profit Trailing System

7 trail modes with dynamic parameter panel

Mode	Description	Parameters
Trail Ticks	Fixed N-tick distance, moves every tick	Trail Ticks
BE +N	One-shot: when up X ticks, stop → entry+N	Trigger, Buffer
Bar L/H	Stop follows prior bar low/high	None
N-Bar L/H	Stop follows N-bar lowest low / highest high	Lookback Bars
Trail ATR	Stop trails at price ± (mult × ATR(period))	ATR Period, Multiplier
Trend Magic	ATR trail gated by CCI regime filter — freezes if CCI doesn't confirm	ATR Period, Multiplier
Half + BE	When up X ticks: close half, move stop to entry+N	Trigger, Buffer

- Only one trail mode active at a time — activating a new mode deactivates the previous
- Dynamic parameter panel — only relevant fields shown per active mode, all editable at all times
- **OnMarketData** override — trail executes on every Last tick in realtime

- **MoveOrPlaceStop()** — finds existing stops (bracket or naked) and moves via `acct.Change()`; places new stop if none exists
- Never moves stop against position — `trailStopLevel` tracks the best stop achieved
- **Trend Magic detail** — `CCI>0` required for long trail, `CCI<0` for short; stop freezes when regime doesn't confirm
- ARM button reset — arming MQB/MQS resets all trail state for clean start per trade
- Scope: chart instrument only

V1.2.2 — Dynamic Scroll Height

Fixes panel clipping at bottom on smaller monitors

- **UpdateScrollViewerHeight()** — calculates rows above the panel, sets `MaxHeight` to exactly remaining available space
- **OnCtGridSizeChanged** — wired to `_ctTraderGrid.SizeChanged`; auto-updates on window resize or dock change
- **OnPanelMouseWheel** — wired to `PreviewMouseWheel` (tunneling phase); intercepts wheel events before NT chart consumes them for zooming
- Removed fixed `MaxHeight=700` cap — replaced by dynamic calculation
- `CanContentScroll=false` — smooth pixel scrolling instead of item-based jumping
- `_ctScrollRow` tracked as field — `RemovePanel()` removes exact `RowDefinition` rather than blindly popping last
- Rebuild button: `Style=null` strips NT default style inflating header row; reduced to 18x18px

V1.3 — Session Risk Management

P&L Max, Drawdown Limit, on-chart risk card

Panel SESSION RISK Section

- New collapsible SESSION RISK section at top of panel above Signal Arming
- **P&L; Max (\$)** text box — enter profit target; 0 = disabled
- **DD Limit (\$)** text box — enter max loss as negative number (e.g. -500); 0 = disabled
- **Flatten on DD** toggle button — arms auto-flatten on DD hit (red glow when active)
- **Apply Limits** button — reads text boxes and applies limits live; triggers unlock check
- Live P&L; display row — shows session P&L; in green/red with MAX HIT / DD HIT status label

On-Chart Risk Card

- Rendered via `OnRender` using `SharpDX Direct2D` — same engine NT8 uses natively
- Only visible when at least one limit is non-zero
- Corner selectable via `RiskCardCorner` property: `TopLeft`, `TopRight`, `BottomLeft`, `BottomRight`
- Shows: SESSION RISK header, P&L; line (green/red), MAX line (amber when hit), DD line (red when hit)
- **Border color communicates state at a glance** — dim gray (active), amber (MAX hit), red (DD hit)

Risk Logic

Property	Type	Default	Description
<code>SessionPnLMax</code>	double	0	Profit target in \$. 0 = disabled

Property	Type	Default	Description
<code>SessionDDLlimit</code>	double	0	Max loss in \$ (negative). 0 = disabled
<code>FlattenOnDDLlimit</code>	bool	false	Flatten ChartTrader account on DD hit
<code>RiskCardCorner</code>	enum	BottomRight	Corner for on-chart display
<code>SoundFile</code>	string	Alert1.wav	Path to .wav file; blank = NT built-in default

- **CheckSessionRisk()** called on every realtime bar via OnBarUpdate
- Uses cachedAccount field — safe from data thread, no Dispatcher.Invoke() needed
- **Lock/unlock** — raising limit above current P&L; auto-unlocks; setting to 0 disables entirely
- **DisarmAll()** — disarms both MQB and MQS; uses Dispatcher.CheckAccess() for thread safety
- **PlayRiskAlert()** — one-shot sound; default Alert1.wav with optional SoundFile override
- **FlattenChartAccount()** — flattens strictly the ChartTrader selected account and instrument only
- Cross-thread fix — account cached on UI thread at panel creation; P&L; reads are data-thread safe

Potential Future Directions

- Auto-bracket on entry — place stop + target the moment MQB/MQS fires
- Session time filter — only arm during specific trading hours
- Entry filters — gate arming on TrendArchitect regime state
- Live P&L; on chart card — running trade count, win/loss tally
- Multi-instrument scope option for trailing and risk

TrendArchitectMQPanel is an open-source project. All contributors credited above. Built for NinjaTrader 8.